Tutoriel des bloquants

Ce qui empêche d'avancer, pourquoi, et quoi changer — dans l'ordre.

État au 10 novembre 2026. À lire avec `docs/todos.pdf` ouvert à côté.

Table des matières

- 0. Pourquoi un tutoriel séparé
- 1. D2 — Le test qui passe à vide
- 2. D3 — Trois fichiers de test jamais compilés
- 3. A1 — Le snapshot : le seul endroit où la base entre dans le moteur
- 4. A3 — Les tables de calibration (migration 010)
- 5. A4 — Le JSON du run
- 6. A5 — L'insertion en base
- 7. A7 — La rejouabilité
- 8. B1 — Quelle convention publie Deribit, vraiment
- 9. B2 — Le kernel retenu
- 10. C2 — L'écart linéaire / inverse, avant d'écrire une PDE
- Annexe — La séquence, en une page

0. Pourquoi un tutoriel séparé

Un backlog dit *quoi* faire. Il ne dit pas *pourquoi ça bloque*, et c'est la seule chose qui compte quand on reprend un projet trois semaines plus tard.

Ce document prend les tâches **BLOQUANT** une par une, dans l'ordre où elles doivent être faites, et pour chacune :

1. **ce qui se passe aujourd'hui** — l'état mesuré, pas l'idée qu'on s'en fait ;
2. **pourquoi ça bloque** — ce qu'on ne peut pas mesurer tant que ce n'est pas fait ;
3. **le schéma** — le flux de données avant / après ;
4. **les maths** — étape par étape, avec les formules et l'ordre de grandeur ;
5. **où changer** — fichier, fonction, signature ;
6. **comment savoir que c'est fini** — un chiffre, pas une impression.

L'ordre n'est pas négociable : chaque chapitre utilise le précédent.

```
   D2  test SSVI qui passe a vide.............|  sans eux, « le test passe »
   D3  trois tests jamais compiles............|  ne prouve rien
                                              |
                                              v
   A1  snapshot base -> moteur................|  sans lui, le moteur ne voit
   A3  schema 010 (run, slice, quote).........|  aucun marche reel : tout le
   A4  JSON du run............................|  reste est theorique
   A5  insertion en base......................|
   A7  rejouabilite...........................|
                                              |
                                              v
   B1  quelle convention publie Deribit.......|  sans elle, chaque prix
   B2  le kernel retenu.......................|  calcule est decoratif
                                              |
                                              v
   C2  ecart lineaire vs inverse               -> decide la coordonnee de Dupire
```

---

1. D2 — Le test qui passe à vide

Ce qui se passe aujourd'hui

`tests/test_ssvi_synthetic.cpp` affiche « TODO » et sort avec le code 0. `ctest` annonce donc :

```
Test #3: test_ssvi_synthetic ..............   Passed    0.17 sec
100% tests passed, 0 tests failed out of 3
```

Deux tests sur trois existent vraiment. Le troisième occupe une ligne de succès.

Pourquoi c'est bloquant

Tous les chapitres qui suivent ont un critère du type « les paramètres sont retrouvés », « l'aller- retour tient à 1e−12 », « le fit reproduit la vue ». Ces critères se vérifient **avec des tests**. Si le harnais de test est décoratif, chaque critère suivant est une opinion.

C'est aussi le seul endroit où le projet peut te mentir sans que personne ne s'en aperçoive : un test vert qui ne teste rien est plus dangereux qu'un test absent, parce qu'il occupe la place.

Ce que doit faire un test d'aller-retour

```
   (rho, eta, gamma, nu)  ⟶  w(k,T)  ⟶  sigma = sqrt(w/T)  ⟶  calibration  ⟶  (rho', eta', ...)
        vrais                                                          doit rendre les vrais
```

Le schéma est volontairement pauvre : aucune donnée de marché, aucun forward, aucune IV de place. Si ce test échoue, le bug est **dans le modèle**, pas dans le pipeline. C'est ce qui en fait un test utile : il isole une seule cause possible.

Les maths, étape par étape

Étape 1. Choisir des paramètres vrais admissibles. Rappel des contraintes du code :

```
rho in [−0.99, 0.99]        eta > 0 (borne haute = eta_max, voir etape 3)
gamma in [0.01, 0.5]        nu in [0.001, 1.0]
```

Exemple qui marche : `rho = −0.35`, `eta = 0.70`, `gamma = 0.30`, `nu = 0.045`.

Étape 2. Construire la variance totale sur une grille :

```
theta(T) = nu * T
phi(theta) = eta / (theta^gamma * (1+theta)^(1-gamma))
w(k,T) = theta/2 * ( 1 + rho*phi*k + sqrt( (phi*k + rho)^2 + 1 - rho^2 ) )
```

Étape 3. Vérifier l'admissibilité — c'est le piège de ce test. La borne n'est pas `eta < inf`, c'est `eta < eta_max(gamma, rho)`, et le code optimise en fait `u = eta / eta_max ∈ [0.01, 1−marge]`. Si tu prends des paramètres vrais hors domaine, la calibration ne *peut pas* les retrouver et tu passes une heure à chercher un bug inexistant.

```
eta_max = min( 4/(1+|rho|) , 2/sqrt( a*r*(1+|rho|) ) )    avec a=1−|rho|, r=0.5−gamma
```

Pour (rho, gamma) = (−0.35, 0.30) : `eta_max ≈ 2.62`, donc `eta = 0.70` donne `u ≈ 0.27`, bien à l'intérieur. Contrôle : `isArbitrageFree(vrais) == true`.

Étape 4. Passer de `w` à `sigma` — et pas l'inverse :

```
sigma(k,T) = sqrt( w(k,T) / T )
```

Étape 5. Calibrer sur cette matrice d'IV et comparer. Les tolérances doivent refléter `xtol_rel` = 1e−8 de BOBYQA, pas la précision machine : viser 1e−12 sur `rho` serait un test qui échoue pour une raison qui n'est pas un bug.

Où changer

`tests/test_ssvi_synthetic.cpp` uniquement. Rien dans la bibliothèque.

Comment savoir que c'est fini

```
[ ] le test affiche les parametres retrouves et l'erreur maximale
[ ] il echoue si l'on inverse deux parametres a la main (controle negatif)
[ ] la tolerance est justifiee par xtol_rel, ecrite en commentaire
[ ] ctest reste a 3/3, mais 3 tests qui testent quelque chose
```

Le **contrôle négatif** est la partie qu'on oublie toujours : un test qui ne peut pas échouer n'est pas un test. Inverse `rho` et `gamma` dans la comparaison, vérifie qu'il casse, puis remets.

---

2. D3 — Trois fichiers de test jamais compilés

Ce qui se passe aujourd'hui

```
tests/test_implied_vol.cpp      pas de main()  ->  jamais compile
tests/test_market_data.cpp      pas de main()  ->  jamais compile
tests/test_pde_pricer.cpp       pas de main()  ->  jamais compile
```

Le CMake ne construit que les fichiers contenant `int main(`. Ces trois-là sont dans le dépôt, donnent l'impression d'une couverture, et ne sont jamais exécutés.

Pourquoi c'est bloquant

`ImpliedVol` a déjà été corrigé une fois (bornes d'arbitrage, bracket). Le correctif n'est protégé par **aucun** test. La prochaine personne qui « simplifie » le solveur peut supprimer le bracket, et rien ne le dira — c'est exactement ce qui s'était passé avec le Newton nu.

Où changer

Ajouter un `main()` à chacun, sur le modèle de `test_pipeline_synthetic.cpp` :

```
int main() {
    // 1. cas nominal
    // 2. cas limite (aile, T court)
    // 3. cas qui doit echouer (prix sous l'intrinseque -> NoArbitrageViolation)
    return failures == 0 ? 0 : 1;
}
```

Le troisième cas est celui qui compte : il vérifie que l'erreur est **distinguée**, pas seulement qu'elle existe.

Comment savoir que c'est fini

```
[ ] ctest liste 6 tests (3 existants + 3 nouveaux)
[ ] chacun contient au moins un cas qui echoue si l'on casse la fonction expres
```

---

3. A1 — Le snapshot : le seul endroit où la base entre dans le moteur

Ce qui se passe aujourd'hui

```
   base TimescaleDB                    moteur C++
   ────────────────                    ──────────

   silver.future_mark  ┐
   silver.option_mark  ├──► gold.v_surface ──►  ... rien ...
   gold.v_forward      ┘

   data/csv/SPX/quotes.csv ──────────────►  MarketData::load  ──►  calibration
   (un fichier figé, sans forward, sans lien avec la base)
```

Le moteur ne sait pas lire la base, et la base ne sait pas produire ce que le moteur attend. Les deux moitiés du projet ont chacune des tests verts, et ne se sont jamais parlé.

C'est **le** blocage : tout le reste (stockage du run, rejouabilité, convention de prix, PDE) suppose un objet `SurfaceSnapshot` qui vienne réellement du marché.

Le flux après A1



Les maths, étape par étape

Étape 1 — le moneyness. `k = ln(K / F(T))`, et non `ln(K/S)`. Pourquoi : mesuré sur BTC le 10/11, la base vaut 489 bp à 10,5 mois, et l'erreur de skew qu'elle induit est de 0,45 point de vol. Le forward est celui de l'échéance, lu sur les futures (`gold.v_forward`).

Étape 2 — combler les échéances sans future. Deribit cote des options quotidiennes mais des futures seulement sur les échéances hebdomadaires/mensuelles/trimestrielles. Entre deux nœuds :

```
ln F(T) = ln F(T1) + (T − T1)/(T2 − T1) ∗ (ln F(T2) − ln F(T1))
```

C'est un **carry constant par morceaux** : `c = ln(F/S)/T` est constant entre deux nœuds. Extrapolation plate au-delà du dernier nœud, et on tronque la grille T plutôt que d'inventer un forward.

Étape 3 — choisir le côté. À chaque strike, on garde l'option la moins chère en valeur relative, c'est-à-dire :

```
k > 0  ->  call   (OTM)
k < 0  ->  put    (OTM)
```

Raison : le prix d'une option profonde dans la monnaie est dominé par l'intrinsèque, et la volatilité implicite y est numériquement instable (vega → 0). Le côté OTM garde la totalité de la valeur temps.

Étape 4 — la grille commune. SSVI ajuste `(rho, eta, gamma, nu)` **simultanément** sur toutes les maturités : il faut donc une seule grille `k` pour toutes. On prend l'intersection des plages couvrées, puis on rééchantillonne :

```
k_grille = k_lo + j ∗ (k_hi − k_lo)/(N−1),  j = 0..N−1,  N = 11 (comme le runner SPX)
```

Étape 5 — l'interpolation, en variance totale et non en vol. C'est le point le plus important de l'étape, et celui qui se rate le plus souvent :

```
w(k,T) = sigma(k,T)^2 ∗ T
```

On interpole `w` linéairement en `k`, puis on revient à `sigma = sqrt(w/T)`. Pourquoi : `w` est presque linéaire en `k` (SVI est une hyperbole en `k`), `sigma` ne l'est pas. L'erreur d'une interpolation linéaire est en `O(h² ∗ w'')` — sur `sigma`, la dérivée seconde est bien plus grande, et l'erreur se concentre dans les ailes, exactement là où le fit est déjà fragile.

Étape 6 — les poids. `w_ij = 1 / vega(k_i, T_j)`, normalisés par leur moyenne (voir le chapitre sur les poids : le calibrateur les consomme, il ne les choisit pas).

Où changer

```
include/model/SurfaceSnapshot.hpp      contrat : deja fait (check(), 11 cas testes)
include/data/SnapshotFile.hpp          le LECTEUR : trois CSV -> structures brutes
src/data/SnapshotFile.cpp
include/model/SnapshotBuilder.hpp      le CONSTRUCTEUR : pur, sans I/O
src/model/SnapshotBuilder.cpp
python/jobs/export_snapshot.py         la base -> le dossier du snapshot
```

Règle de répartition, à ne pas mélanger : `data/` fait de l'entrée/sortie, `model/` fait du calcul. Le builder ne filtre rien (la propreté est garantie par la base) ; ses seuls contrôles sont les `assert` du contrat.

Comment savoir que c'est fini

```
[ ] k recalcule en C++ = k_fwd de la vue, a 1e−12, sur les echeances a future
[ ] 11 echeances et >= 3000 quotes sur BTC
[ ] un test synthétique : forward connu -> F retrouve, paramètres retrouves
[ ] check() == Ok sur le snapshot produit, et le test refuse un snapshot casse
```

---

4. A3 — Les tables de calibration (migration 010)

Ce qui se passe aujourd'hui

`calibration_run`, `calibration_slice`, `quote_fit` ont été **supprimées par la migration 004**, en même temps que l'expérience CBOE. Le commentaire de 004 dit pourquoi : leur forme était taillée pour des options sur indice (`FK` vers un snapshot SPX, strikes en points, colonnes `iv_cboe`).

Conséquence aujourd'hui : un fit ne laisse aucune trace. On lit un JSON dans un terminal, et le lendemain on ne sait plus quel fit a produit quel chiffre.

Pourquoi c'est bloquant

A7 (rejouabilité), E4 (surveillance du fit) et la comparaison de deux conventions (B1) supposent tous de pouvoir **relire un run**. Sans table, chaque mesure est un one-shot non reproductible.

La leçon de 004, à ne pas répéter

Le schéma précédent portait des hypothèses de marché dans le schéma : des strikes `NUMERIC` en points d'indice, une FK vers un snapshot d'une seule source. Un schéma qui encode le marché se casse à chaque changement de marché.

Trois colonnes évitent 90 % des regrets :

```
snapshot_id     ce qui a été ajuste (et non "la base au moment du run")
engine_git_sha  ce qui a calcule (un binaire non trace n'est pas un resultat)
conventions     linear_usd | inverse_coin (le choix de B1, ecrit dans la ligne)
```

Où changer

`sql/010_calibration.sql`, dans l'ordre : `surface_snapshot`, `calibration_run`, `calibration_slice`, `quote_fit`. Contraintes minimales :

- `calibration_run.run_id` PK, `snapshot_id` NOT NULL, `engine_git_sha` NOT NULL ;
- `calibration_slice` PK `(run_id, expiry)`, FK vers `calibration_run` ;
- `quote_fit` en hypertable sur `observed_at`, PK `(run_id, expiry, strike, is_call)` ;
- `surface_snapshot` : la grille et le chemin du fichier, pour que `snapshot_id` ne soit pas un identifiant opaque.

Comment savoir que c'est fini

```
[ ] 001 -> 010 sur une base neuve, sans erreur, et le migrateur laisse schema_migrations a 010
[ ] 009 -> 010 sur la production existante (la base courante), sans divergence
[ ] rejouer 010 echoue proprement (deja appliquee), ne casse rien
```

---

5. A4 — Le JSON du run

Pourquoi un fichier intermédiaire

Le C++ n'a pas de client Postgres, et lui en donner un créerait une dépendance réseau dans un binaire de calcul. Le JSON est la frontière : le C++ écrit un artefact complet, Python l'insère.

Le contenu minimal

```
{
  "schema_version": 1,
  "snapshot_id": "deribit-BTC-20261110T195000Z",
  "engine_git_sha": "360b003",
  "conventions": "inverse_coin",
  "started_at": "2026-11-10T20:05:11Z",
  "finished_at": "2026-11-10T20:05:19Z",
  "status": "ok",
  "params": { "rho": -0.42, "eta": 1.31, "gamma": 0.28, "nu": 0.061 },
  "diagnostics": {
    "rmse_total": 0.0041, "max_abs_error": 0.012, "eta_ratio": 0.61,
    "slices": [ { "T": 0.12, "n_points": 11, "rmse": 0.0032, "max_abs_error": 0.009 } ]
  }
}
```

Pourquoi `schema_version`

Un JSON sans version devient illisible dès la première colonne ajoutée : Python ne sait plus si `diagnostics.rmse_total` est en points de vol ou en variance. Le numéro permet de refuser un fichier ancien au lieu de l'interpréter de travers.

Où changer

`apps/calibrate_deribit.cpp` (le runner, à écrire à partir de `calibrate_spx_minimal.cpp`). L'écriture du JSON est la seule partie « I/O » : le reste appelle la bibliothèque.

Comment savoir que c'est fini

```
[ ] python -c "import json; json.load(open('run.json'))" passe
[ ] le JSON contient snapshot_id et engine_git_sha non vides
[ ] recompiler sans changer de commit ne change pas le JSON (hors horodatages)
```

---

6. A5 — L'insertion en base

Le point délicat : les résidus

Le C++ calibre sur une **grille** rééchantillonnée (A1, étape 4). Écrire `quote_fit` depuis cette grille donnerait 11 lignes par échéance — pas un résidu par cotation.

On recalcule donc, côté Python, l'IV modèle **aux strikes réels**, depuis les 4 paramètres. C'est possible parce que SSVI est en forme fermée :

```
theta  = nu * T
phi    = eta / (theta^gamma * (1+theta)^(1-gamma))
w      = theta/2 * (1 + rho*phi*k + sqrt((phi*k + rho)^2 + 1 - rho^2))
sigma  = sqrt(w / T)
```

Puis le résidu par cotation : `residual = sigma_modele - iv_marche`, avec `k = ln(K/F(T))`.

L'idempotence, sans laquelle un cron fabrique des doublons

```
insert into calibration_run (...) values (...) on conflict (run_id) do nothing
```

où `run_id` est déterministe : `sha1(snapshot_id | engine_git_sha | conventions)`. Deux exécutions sur le même snapshot ne créent qu'une ligne, et une recalibration volontaire change le sha du binaire — donc crée un nouveau run, ce qui est le comportement voulu.

Où changer

`python/jobs/calibrate.py` : lire le JSON, insérer `calibration_run`, `calibration_slice`, puis `quote_fit` en `COPY` (comme `load_marks.inserer`). Une transaction.

Comment savoir que c'est fini

```
[ ] un run complet en base, relu par un select
[ ] rejouer le meme JSON n'ajoute aucune ligne
[ ] le nombre de lignes de quote_fit egal au nombre de quotes du snapshot
```

---

7. A7 — La rejouabilité

Pourquoi ce n'est pas automatique

Un fit n'est reproductible que si **les trois** sont figés : la donnée, le code, et l'ordre des opérations.

Source de non-déterminisme	Ce qui la révèle	Le remède
ré-exporter le snapshot à un autre instant	deux fits « identiques » diffèrent	`snapshot_id` fige les fichiers, le builder ne lit jamais `now()`
itérer sur un `unordered_map` d'instruments	l'ordre des quotes change, la somme des carrés change au dernier bit	vecteurs triés, ordre total explicite
un binaire différent du sha enregistré	paramètres différents, même run_id	`engine_git_sha` dans la clé

BOBYQA est déterministe à point de départ fixe : pas de graine aléatoire à gérer, mais le point de départ doit être écrit en dur (`{-0.5, 1.0/etaMax(0.3,-0.5), 0.3, 0.04}`), pas tiré au hasard.

Comment savoir que c'est fini

```
[ ] deux executions a 24 h d'intervalle, meme snapshot_id -> parametres identiques a 1e-10
[ ] diff des deux JSON, hors started_at/finished_at : vide
```

Si les paramètres diffèrent de 1e−9, ce n'est pas grave ; s'ils diffèrent de 1e−3, il y a un ordre d'itération non maîtrisé quelque part.

---

8. B1 — Quelle convention publie Deribit, vraiment

Le problème en une phrase

On ajuste une surface en supposant que l'IV publiée est celle d'un modèle donné. Si ce n'est pas le bon modèle, on ajuste une surface qui ne reproduit aucun prix.

Les maths

Modèle (a) — linéaire, réglé en dollars. C'est Black-76, ce que fait le moteur aujourd'hui :

```
C_usd = D * [ F*N(d1) - K*N(d2) ]
d1 = [ ln(F/K) + 0.5*sigma^2*T ] / (sigma*sqrt(T)),   d2 = d1 - sigma*sqrt(T)
```

Modèle (b) — inverse, réglé en coin. Le payoff d'un call, en BTC, est `(1 - K/S_T)^+`.

Sous le numéraire coin, la quantité `X = 1/S` est une martingale (le dollar exprimé en BTC n'a pas de dérive). Donc :

```
C_coin = E_coin[ (1 - K*X_T)^+ ] = N(d1) - (K/S0)*N(d2)
```

avec le **même** `d1 = [ln(S0/K) + 0.5σ²T]/(σ√T)` : la volatilité est celle de `X`, et comme `X = 1/S` a la même volatilité que `S`, c'est aussi celle de `S`.

La parité, qui départage tout de suite. En soustrayant call et put :

```
C - P = E_coin[1 - K/S_T] = 1 - K*E_coin[1/S_T] = 1 - K/S0
```

Le strike où `C = P` vaut `S0`, le **spot**. Pas le forward. Vérifié numériquement (3 M de tirages, σ=40 %, T=0,5, base 490 bp) : croisement à 73 000 pour un spot de 73 000 et un future à 74 811.

Conséquence 1. En actions, la parité donne `F` ; en options inverses, elle donne le spot. Ce sont deux mondes différents, et c'est pour ça que les recettes de place ne se transposent pas.

Conséquence 2 — et c'est une correction de ce que j'annonçais. J'avais listé trois modèles (a) linéaire, (b) inverse, (c) put linéaire sur `1/S`. Les maths en donnent **deux** : car

```
(1 - K*X)^+ = K * (1/K - X)^+
```

donc (c) est exactement (b) réécrit — un put sur `X` de strike `1/K`, multiplié par `K`. Ce n'est pas un troisième candidat, c'est un moyen de tester (b) : implémente les deux, ils doivent coïncider à 1e−12.

Le test qui tranche

Sur ≥ 5 000 quotes et ≥ 10 échéances :

```
pour chaque quote (K, T, mark_iv, mark_price) :
    (a) prix_a = black76(F(T), K, T, mark_iv/100, D=1)
    (b) prix_b = inverse(S0, K, T, mark_iv/100)
    erreur = (prix_calcule - mark_price) / mark_price

    -> tableau par convention, par echeance, ATM vs ailes
```

Celle qui colle à moins d'un tick gagne. Attention aux unités : `mark_iv` est en **pourcentage** dans la base, `mark_price` en **coin** — deux pièges déjà rencontrés sur ce projet.

Où changer

`tools/check_convention.py` (nouveau, aucune modification du moteur). Le test lit la base, calcule, écrit un tableau. Il ne modifie rien : c'est une mesure.

Comment savoir que c'est fini

```
[ ] le tableau existe, avec un ecart median et maximal par convention et par echeance
[ ] (b) et (c) coincident a 1e-12 (validation de l'implementation)
[ ] une convention gagne clairement, ou l'ecart est explique
```

---

9. B2 — Le kernel retenu

Ce qu'on écrit

Selon B1. Si c'est (b) — le plus probable — un `InverseKernel` qui implémente l'interface déjà existante :

```
price(isCall, F, K, T, df, sigma)   // ici F = S0, df = 1
iv(isCall, px_coin, F, K, T, df)
```

Les bornes d'arbitrage, en unités coin

C'est le piège du portage : les bornes de `ImpliedVol` sont exprimées en dollars. En coin, elles deviennent :

```
call : max(1 - K/S0, 0)  <=  C_coin  <=  1
put  : max(K/S0 - 1, 0)  <=  P_coin  <=  K/S0
```

Un call vaut au plus 1 BTC (payoff borné par 1), un put au plus `K/S0` BTC. Confondre les deux unités donne un solveur qui refuse toutes les quotes des ailes.

Comment savoir que c'est fini

```
[ ] aller-retour prix -> IV -> prix a 1e-12, sur ATM et sur les ailes
[ ] les prix reproduits a un tick pres contre mark_price (B1)
[ ] un prix sous l'intrinseque rend NoArbitrageViolation, pas une valeur
```

---

10. C2 — L'écart linéaire / inverse, avant d'écrire une PDE

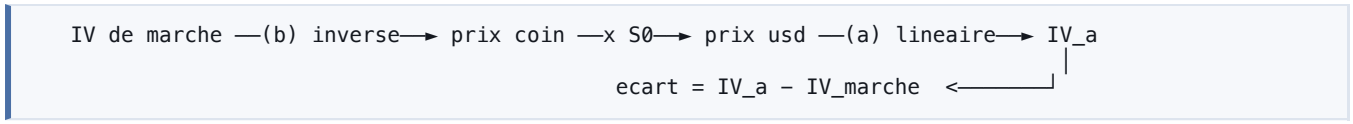
Pourquoi mesurer avant de coder

Écrire une PDE en `1/S` coûte une semaine. Si l'approximation linéaire est à 20 bp de prix sur les ailes, personne ne la verra jamais — et la semaine est perdue. Si elle est à 2 %, la PDE est obligatoire. La mesure coûte une heure.

Le protocole

Pour une même quote :

1. calculer le prix coin avec le modèle retenu (b), à l'IV de marché ;
2. convertir en dollars : `P_usd = S0 * P_coin` ;
3. donner ce prix au modèle (a) et retrouver son IV implicite ;
4. `ecart = IV_a - IV_marche`, en points de vol.



Le seuil de décision

écart sur les ailes	conséquence
< 0,1 pt de vol	on garde le moteur linéaire, on documente l'approximation
0,1 à 0,5 pt	on documente, et la PDE devient un choix de précision, pas de correction
> 0,5 pt	la PDE dans la bonne coordonnée est obligatoire avant tout pricing

Pour situer : l'erreur qu'on vient de corriger sur `k` valait 0,45 pt de vol à 10 mois. L'ordre de grandeur de l'écart de convention est du même type ($\sigma^2 T$), donc probablement dans la zone du milieu.

Où changer

`tools/check_convention.py` (même outil que B1, une colonne de plus), puis la décision se note dans le cahier (§5, déc. 2).

Annexe — La séquence, en une page

#	Tâche	Ce qui débloque	Sortie vérifiable
1	D2	« le test passe » devient une information	3 tests qui testent, dont un contrôle négatif
2	D3	le solveur d'IV est protégé	6 tests dans ctest
3	A1	la base entre dans le moteur	k C++ = k de la vue à 1e−12
4	A3	un run peut exister	migration 010 appliquée deux fois sans divergence
5	A4	un run est un artefact	JSON relu, avec snapshot_id et sha
6	A5	un run est relisible	une ligne en base, rejouable sans doublon
7	A7	un run est reproductible	deux JSON identiques à 1e−10
8	B1	on sait ce qu'on price	un tableau d'écarts qui tranche
9	B2	le moteur price juste	prix à un tick de la place
10	C2	la PDE devient un choix, pas une croyance	un écart chiffré, et la décision écrite

Les étapes 1 et 2 sont courtes (une session chacune) et sans elles les huit autres ne sont pas vérifiables. C'est le meilleur retour sur investissement du backlog, et de loin le moins excitant : c'est exactement pour ça qu'il faut les faire d'abord.